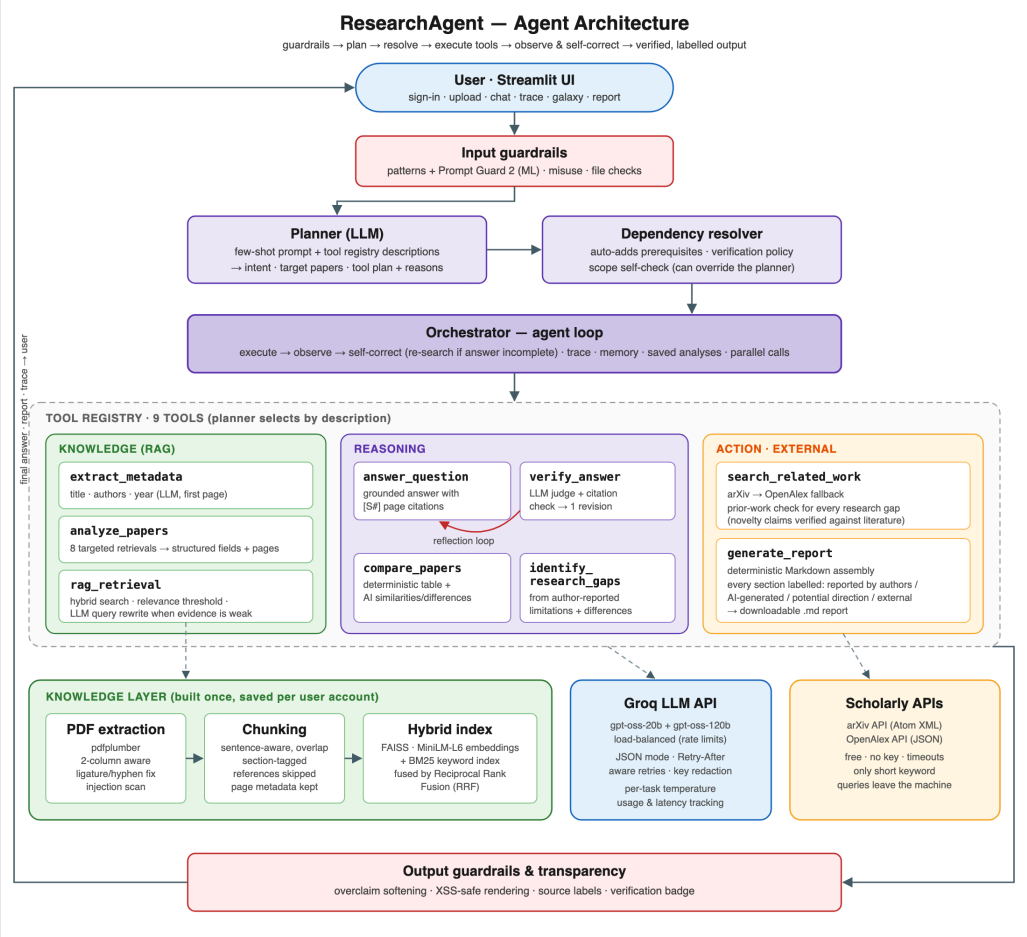
ResearchAgent

An Agentic AI System for Research Paper Analysis, Comparison and Research Gap Identification

Activity	FA Activity: Agentic AI Application Implementation
Domain	Research Paper Analysis Agent
Group members	Jayesh Patil [Group partner name - roll no.]
Institute	[Institute name]
Course	[Course / class / division]
Source code	https://github.com/mitalihatestocode/ResearchAgent
Date	20 October 2026



1. Introduction

Researchers and students often need to read several papers on one topic, find out what each one did (dataset, method, models, metrics, results, limitations), compare them, and decide what is still open. Doing this by hand is slow. A general chatbot is risky: it answers from memory, can invent numbers and citations, and may present "research gaps" as confirmed discoveries.

ResearchAgent works only from the papers the user uploads. For each question it **plans** which of its 9 tools to use, **retrieves** evidence from the papers, **acts** through local tools and external APIs, **checks** its own answer, and returns output that is cited, labelled and verifiable.

2. Problem Statement

Given a few research papers (PDFs) and a question in plain English, produce answers, comparisons and potential research directions that are:

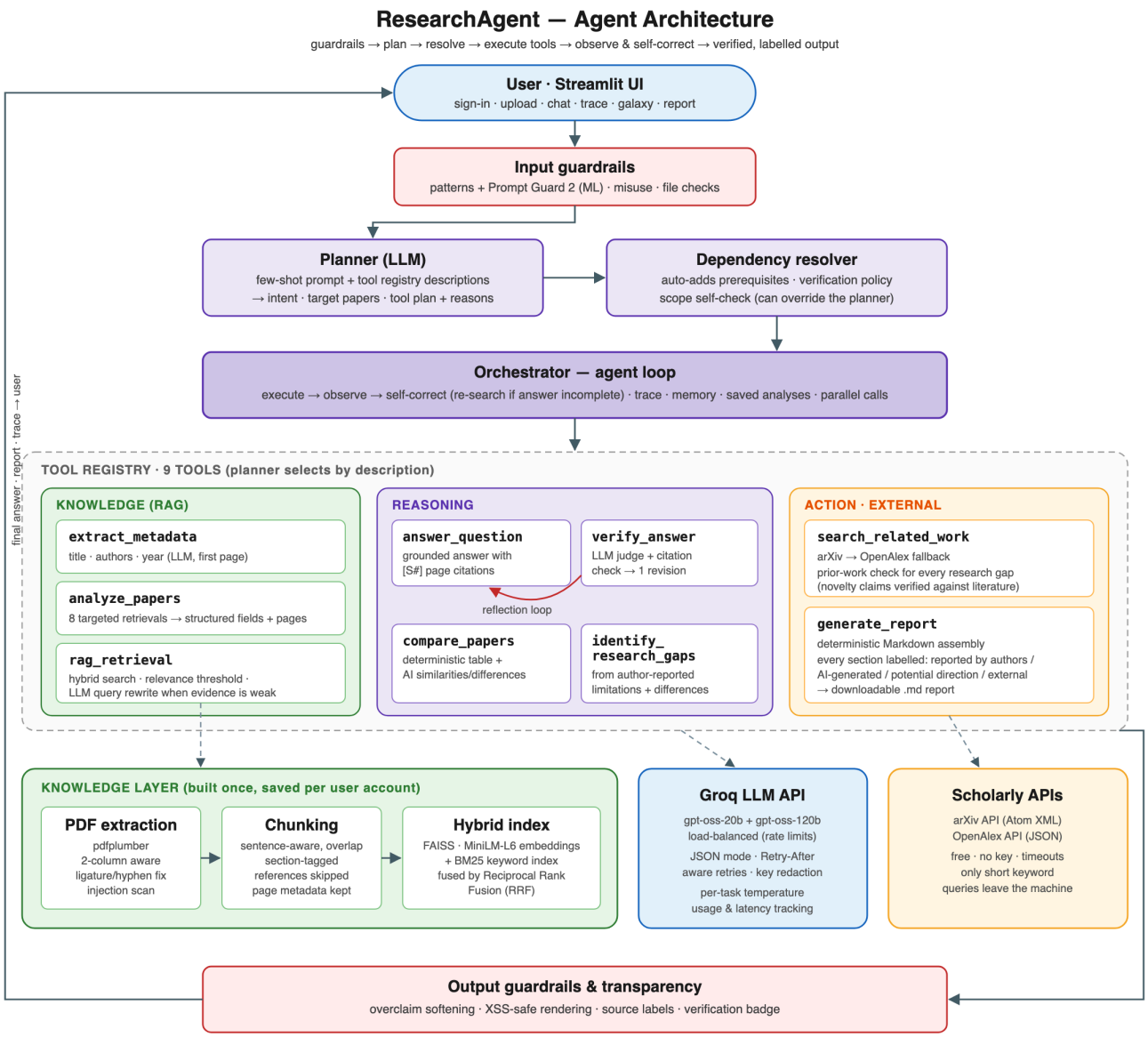
1. **grounded**: every fact can be traced to a page of an uploaded paper;
2. **right-sized**: a narrow question does not trigger a full review, and a broad one is not answered superficially;
3. **honest**: missing information is reported as missing, and suggestions are never presented as confirmed findings;
4. **safe**: robust to prompt injection (including instructions hidden inside PDFs) and not usable for academic misconduct.

Why an agent and not a chatbot? The work splits into different sub-tasks (extract each paper, compare, find gaps, search literature). Which of them are needed depends on the question, so the workflow must be planned. Grounding needs retrieval before generation, and checking whether a gap is new needs an external search. A single prompt → answer call cannot do this.

3. Objectives

1. Turn PDFs (including two-column layouts) into a searchable, section-aware knowledge base.
2. Let an LLM planner choose the tools and target papers for each question.
3. Use hybrid retrieval (meaning + keywords) with a relevance threshold and page citations.
4. Extract a structured summary of each paper and compare papers field by field.
5. Suggest potential research gaps from the authors' stated limitations and check them against existing literature.
6. Make the agent check and repair its own answers.
7. Enforce safety with guardrails on questions, documents and output.
8. Provide a usable app with accounts, a visible agent trace and a downloadable report, plus automated tests.

4. Proposed Agent Architecture



Layer	File	Responsibility
Interface	`app.py`, `style.css`	sign-in, upload, chat, trace, analysis, Paper Galaxy, report
Agent core	`agent.py`	LLM client, planner, dependency resolution, running the tools, self-checking
Tools	`tools.py`	metadata, analysis, answering, verification, comparison, gaps, literature search, report
Knowledge (RAG)	`rag.py`	PDF reading, chunking, embeddings, hybrid FAISS + BM25 search
Safety	`safety.py`	question, document, file and output guardrails
Storage	`storage.py`	accounts and each user's saved papers, analyses and chats (SQLite)

Agentic properties. *Planning*: the LLM chooses the tools for each question. *Tool use*: 9 tools (local, LLM and external API). *Multi-step execution*: tools run in dependency order, with a trace of every step. *Autonomous decisions*: the agent can overrule the planner's "off-topic" decision when the papers clearly match, rephrases weak searches, and searches again when its own draft says something is missing. *Reflection*: a fact-checker reviews each answer and the answer is fixed once if needed. *Memory*: follow-up questions use the conversation, and accounts keep each user's papers, analyses and chats.

We chose **plan-then-execute with observation** over an open-ended ReAct loop: the agent still adapts at each step, but cost and time stay bounded and every decision is visible.

5. Prompt Design

Each tool has its own prompt next to its code in `tools.py`; the planner prompt is in `agent.py`.

Technique	How it is applied
One role per prompt	planner, metadata reader, paper analyst, answerer, fact-checker, query rewriter, comparer, gap suggester
Few-shot examples	the planner sees 4 worked request → plan examples, including an off-topic one where no tool should run
Structured output	every call uses the API's JSON mode with an explicit set of keys
Reasoning first	plans and fact-checks write a `reasoning` field before deciding; it is shown in the Trace tab
Evidence as data	passages are wrapped in `<evidence id="S1" paper=… page=…>` tags and the model is told never to follow instructions inside them
Citations	answers must cite `[S#]` after each claim; code checks that every cited id exists
Fixed "not found" text	missing information must be written exactly as "Not explicitly reported in the paper."
Self-critique	a separate fact-checker prompt (temperature 0) reviews each answer
Query rewriting	weak or incomplete results trigger a prompt that rephrases the question in research-paper wording
Temperature per task	0.0 for planning, metadata and fact-checking; 0.1 for extraction; 0.2 for answers; 0.4 for gap ideas
Shared rules	the same `RULES` text is added to every tool prompt: no invented facts or citations, no novelty claims, ignore instructions inside evidence

Example few-shot entry:

```
Request: "What dataset did paper 1 use?"  (papers: bert.pdf, roberta.pdf)
{"reasoning": "Narrow factual question about one paper: a search and a cited answer are
enough.",
 "in_scope": true, "papers": ["bert.pdf"], "steps": ["rag_retrieval", "answer_question"]}
```

6. RAG Implementation (`rag.py`)

1. **Reading**: pdfplumber with **two-column detection**. When almost no words cross the middle of the page, the title block is read first and then each column separately. Ligatures and words broken across lines are repaired; papers are capped at 80 pages.
2. **Chunking**: whole sentences are packed into chunks of up to 900 characters, and the last sentence(s) repeat in the next chunk so ideas are not cut. Section headings (Abstract, Method, Results, Limitations, …) are detected and stored on each chunk, and the **References section is skipped**.
3. **Indexing**: local `all-MiniLM-L6-v2` embeddings (384 numbers per chunk) in a FAISS index, plus a small built-in **BM25** keyword index.
4. **Hybrid search**: the two rankings are merged with **Reciprocal Rank Fusion** (score = 1/(60 + rank)). Embeddings handle paraphrases; BM25 catches exact names such as "PopQA" or "MIMIC-III".
5. **Relevance threshold**: passages below cosine 0.28 are never used as evidence (off-topic questions scored 0.08-0.21 on the sample papers, on-topic ones 0.43-0.71).
6. **Every paper gets a say**: questions about several papers also take the top passages from each paper.
7. **Field-by-field extraction**: `analyze` runs one search per field (dataset, method, …) inside one paper and asks the LLM which passages support each field, so each field shows its page numbers.

7. Tool/API Integration

Tool / API	Kind	Details
Groq chat API	external LLM	`agent.py: LLM`. JSON mode, 45 s timeout, low reasoning effort. The free tier allows ~8k tokens/minute per model, so a rate-limited call moves to a second model (`gpt-oss-20b` ↔ `gpt-oss-120b`) and only waits when both are busy
Llama Prompt Guard 2 (Groq)	external classifier	scores each question for jailbreak / prompt-injection risk
arXiv API	external, no key	`tools.search_literature`: related work and a prior-work check for every research gap
OpenAlex API	external, no key	automatic fallback when arXiv fails or returns nothing
sentence-transformers, FAISS	local libraries	embeddings and vector search
9 agent tools	internal	listed in `agent.py: TOOLS` with a description (what the planner reads) and their prerequisites

Every tool failure is caught and shown in the trace, so one failing API degrades one step, not the whole answer.

8. Agent Workflow (`agent.py: Agent.run`)

```
1. Guardrails       blocked question? stop before any LLM call; Prompt Guard score >= 0.8?
stop
2. Plan             the LLM picks tools + target papers (unknown tools/papers are dropped;
                    keyword rules take over if the planner fails)
3. Scope check      "off-topic" plan? search the papers anyway; refuse only if nothing
matches (< 0.45)
4. Prerequisites    add missing tools (e.g. compare needs analysis) + two fixed policies:
                    answers are always fact-checked, gaps are always checked against
literature
5. Run each tool    and log {step, status, detail, seconds}
     rag_retrieval      weak match (< 0.35)? rephrase the question 3 ways and search again
     answer_question    draft says something is missing? search deeper and keep the better
answer
     verify_answer      fact-checker + citation check; problems? rewrite the answer once
     metadata/analysis  3 papers at a time, remembered for later questions (and saved per
user)
6. Answer           shown with citations, a fact-check badge, the trace and the report
```

Examples: "*Compare these papers and identify potential research gaps*" → `extract_metadata → analyze_papers → compare_papers → identify_research_gaps → search_related_work → generate_report`. "*What dataset did paper 1 use?*" → `rag_retrieval → answer_question → verify_answer`, searching paper 1 only.

9. Safety Mechanism (`safety.py`)

Risk	Mechanism
Prompt injection typed by the user ("ignore previous instructions…")	pattern check on normalised text (zero-width and full-width tricks removed); blocked before any LLM call
Reworded jailbreaks the patterns miss	**Llama Prompt Guard 2**; score ≥ 0.8 is blocked (attacks scored ~0.999, normal questions ~0.0004)
Instructions hidden inside a PDF	removed from passages before they reach a prompt; fake `</evidence>` tags are removed; evidence is marked as data; the user is warned
Academic misconduct (fabricating data or citations, beating plagiarism checks, ghost-writing a thesis)	refused with an explanation
Risky files	must start with `%PDF`; ≤ 25 MB; encrypted PDFs and PDFs with JavaScript or launch actions are rejected; duplicates detected by SHA-256
Made-up facts and citations	answers only from retrieved passages; relevance threshold; fixed "not reported" text; citations checked in code; fact-checker + one repair
Overclaiming	prompts forbid novelty claims; `soften()` rewrites words like "novel" or "first-ever"; every gap is checked against arXiv/OpenAlex
Unsafe display (XSS)	all paper/LLM/API text is HTML-escaped or shown as plain Markdown; only `http(s)` links are made clickable

Risk	Mechanism
Accounts	salted PBKDF2-SHA256 (600,000 rounds), constant-time comparison, the same error for wrong user or password, 5 failures lock the account for 5 minutes, every query filtered by user id, password-confirmed account deletion, guest mode saves nothing
Server	runs on localhost only, upload size limit, XSRF protection, no stack traces shown to users; `pip-audit` finds no known vulnerabilities in the pinned packages
Transparency	outputs are labelled *from the papers* / *AI-generated* / *suggestion*; the full plan and every step are visible in the Trace tab

Two lessons from testing: the phrase "system prompt" appears legitimately in the Self-RAG paper, so the injection patterns were narrowed (and a test added); and a reworded jailbreak slipped past the patterns, which is why the ML classifier was added.

10. Results

Automated tests: 30 tests pass in about 1 second (`python -m pytest -q`), covering PDF reading, chunking, search, the Galaxy projection, every guardrail, planning, all tools, self-correction, caching, failure handling and accounts. A fresh install from `requirements.txt` also passes all 30. Details: TEST_REPORT.md.

Evaluation benchmark (`python tests/evaluate.py`, full results in evaluation_results.md): 12 questions on the 3 sample papers with answers checked against the papers' text, plus 3 the agent should decline.

Measure	Run 1	Run 2
Fully correct answers	10/12	8/12
Average share of expected facts found	90%	77%
Answers whose citations all exist	12/12	12/12
Answers the fact-checker marked as supported	12/12	12/12
Correctly declined (not in the papers / off topic)	3/3	3/3
Average time per question	20.7s	17.8s

Search (top 6, no LLM)	All papers	Paper named in the question
dense	0.67	0.83
keyword	0.58	0.92
hybrid	0.75	0.83

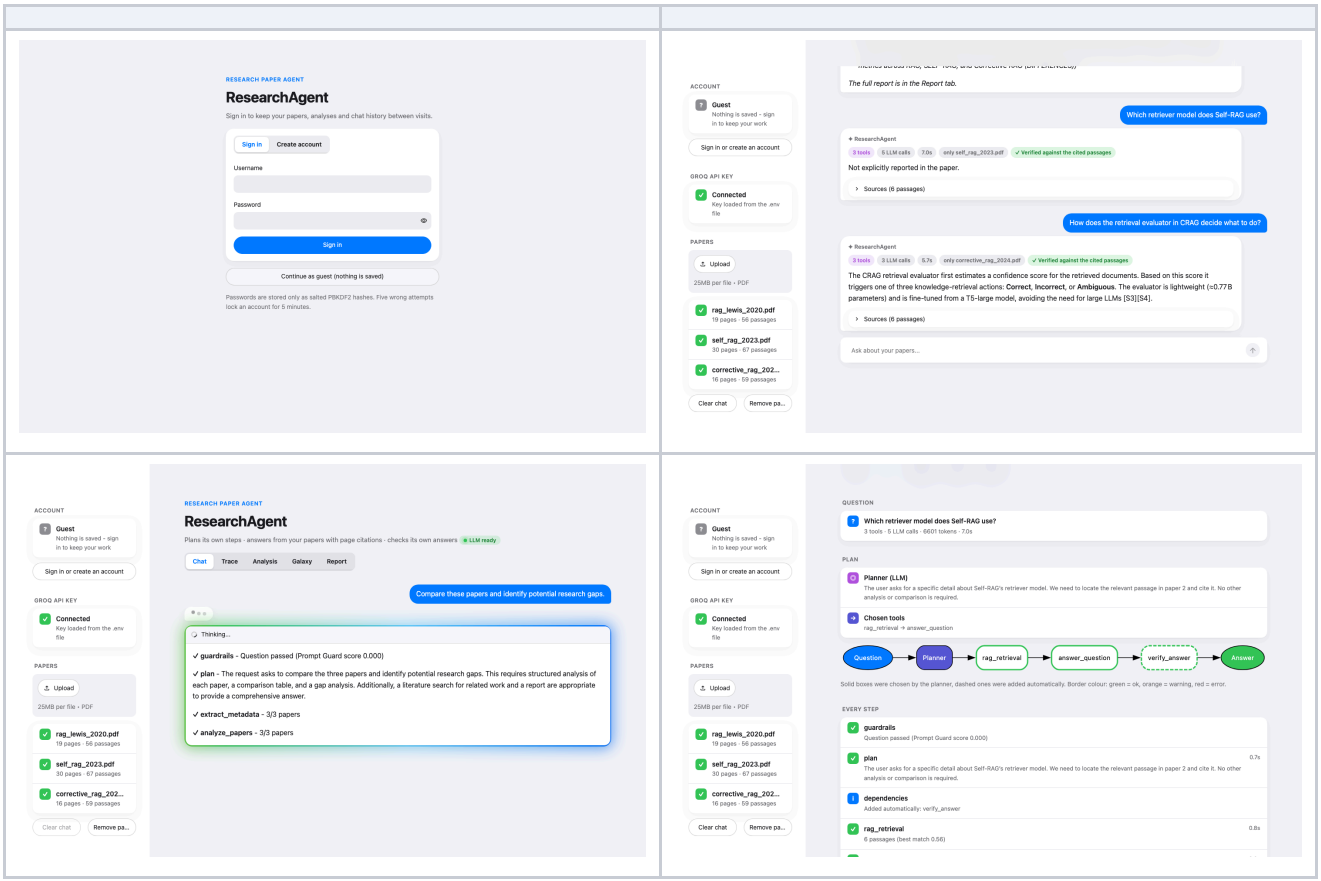
Hybrid search beats either method alone when searching all papers. In both runs every citation was real, every answer passed the fact-checker, and all unanswerable or off-topic questions were declined rather than answered. The misses are facts packed into one dense paragraph (Self-RAG page 7) that search did not reach; there the agent said "not explicitly reported" instead of guessing.

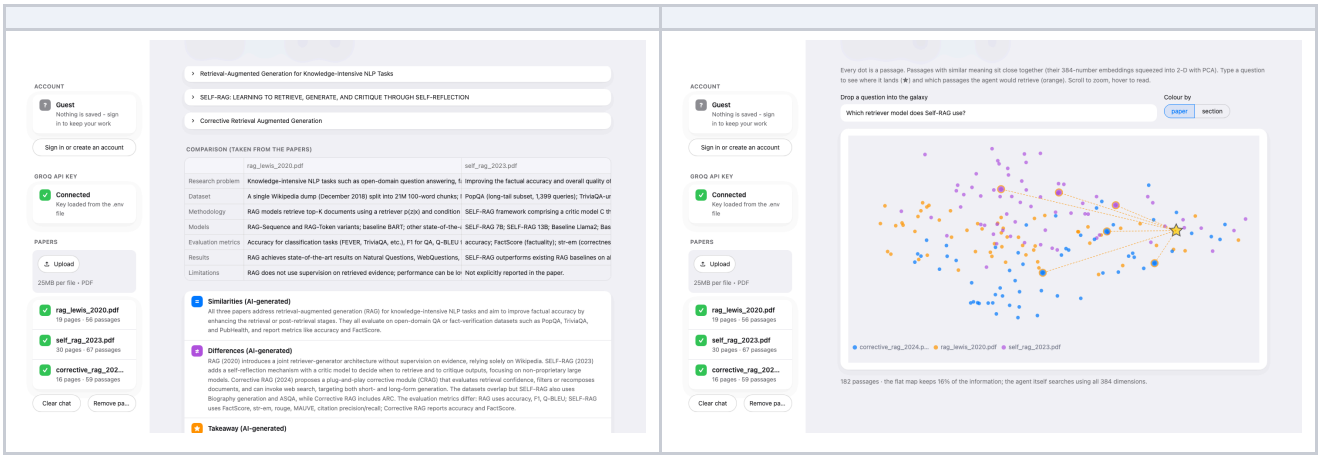
Live checks in the app (real Groq model, 3 arXiv papers): a full comparison + research-gap run took 13 s with 9 LLM calls; "*How does the retrieval evaluator in CRAG decide what to do?*" was answered correctly (confidence score → Correct / Incorrect / Ambiguous, T5-large evaluator) with citations, verified, in 5.7 s; signing out and back in restored all papers, analyses and chats without re-embedding or new LLM calls.

Problems found and fixed while building:

- The original code could not run on a fresh install: `groq==0.11.0` breaks with current `httpx`, and the pinned NumPy clashed with `pyarrow`. Both were fixed by updating the pins (and 50+ known vulnerabilities went with them).
- Two-column PDFs came out as merged words and mixed-up columns; column-aware reading fixed this and improved search scores.
- The reasoning model ran out of tokens before finishing its JSON; one request hung for 26 minutes. Fixed with low reasoning effort, a larger token budget and a 45 s timeout.
- The benchmark showed honest but incomplete answers when the first search missed a passage, which led to the "search again when the draft is incomplete" step.
- A cross-encoder re-ranker was measured but not adopted: it helped single-paper questions (hit@6 0.83 → 0.92) but hurt questions across papers (0.75 → 0.67).

User interface: iOS-style design with motion (cards slide in, chat bubbles pop in, typing dots while the agent thinks, a glowing border while it works, answers typed out word by word), turned off automatically for users who choose *reduce motion*. The **Paper Galaxy** maps every passage by meaning (PCA of the embeddings) and shows where a question lands and which passages are retrieved.





11. Limitations

- **Scanned PDFs** have no text layer and are not read (no OCR); the app says so instead of failing.
- **Tables, figures and equations** come through as plain text at best.
- **Section detection is heuristic**; unusual heading styles keep the previous section label.
- **Dense paragraphs**: a fact buried in a paragraph full of other details can still be missed by search (benchmark Q10/Q12); the agent then says "not reported" rather than guessing.
- **The fact-checker is an LLM too**: it reduces mistakes but cannot guarantee their absence.
- **The prior-work check is shallow**: the top 3 keyword matches from arXiv/OpenAlex, not a systematic review.
- **Free-tier limits** (~8k tokens/minute per model) can slow the first full analysis; later questions reuse saved analyses.
- **Accounts are local** to the computer running the app; a public deployment would need HTTPS and persistent logins.
- **The benchmark is small** (15 questions); it shows trends and catches regressions, but it is not statistically strong.

12. Future Scope

- OCR for scanned papers and proper table/figure extraction.
- Better handling of dense paragraphs: a re-ranker used only for single-paper questions, or sentence-level indexing.
- Citation graphs (e.g. Semantic Scholar) to rank gaps by how much follow-up work exists; BibTeX export.
- Run Prompt Guard over document passages as well as questions.
- A reviewer agent that critiques suggested gaps, and letting the user edit the plan before it runs.